Fitness Company Database System

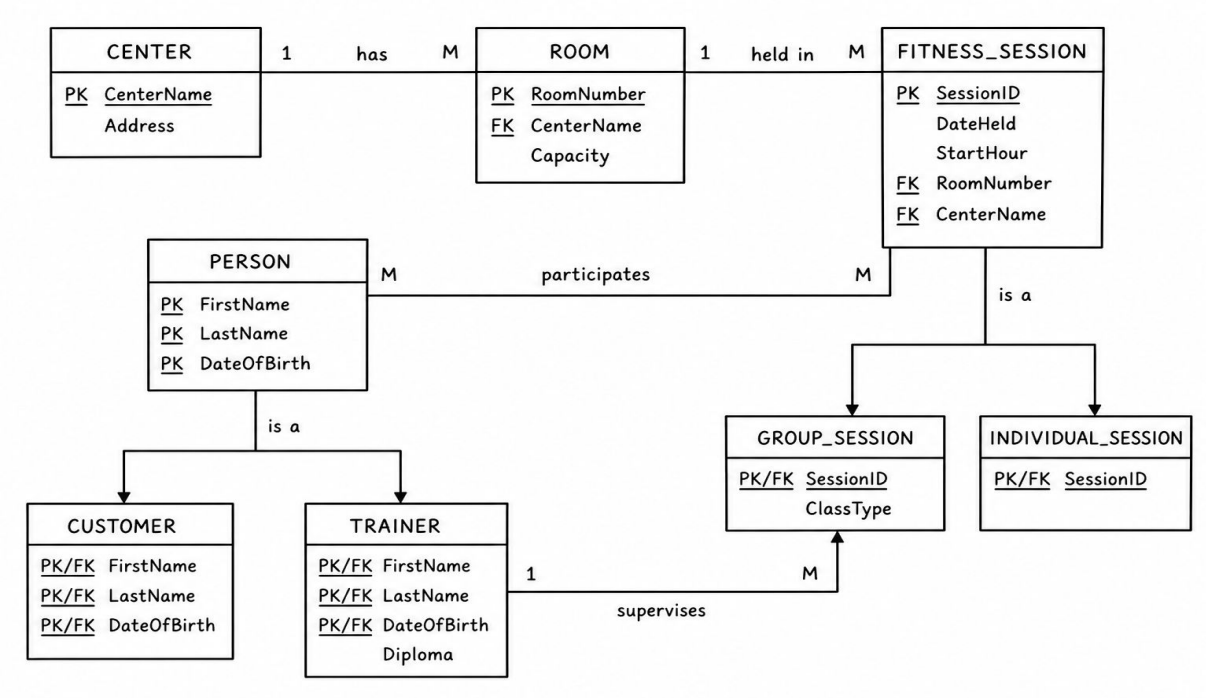
Justin Garrett

CSCI 0340

Project Goal

This project focuses on designing and implementing a database system for a fitness company. The goal is to organize and manage data related to centers, rooms, members, trainers, and sessions.

ER Diagram



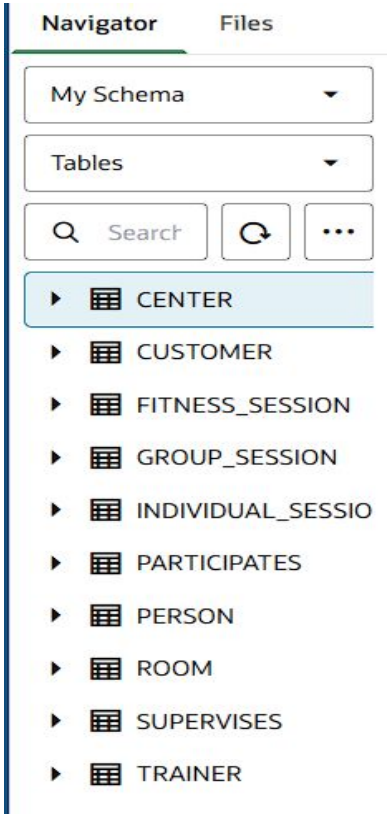
My ER model represents the system at a conceptual level. The main entities are Center, Room, Person, and Fitness Session.

The relationships show how these entities connect. For example, a center has many rooms, and a room can host many sessions.

A person can participate in multiple sessions, creating a many-to-many relationship.

The model also includes specialization, where a person can be either a customer or a trainer, and sessions can be group or individual.

Relational Schema & Implementation



I converted the ER model into a relational schema by creating tables such as CENTER, ROOM, PERSON, TRAINER, CUSTOMER, and FITNESS_SESSION.

I also created supporting tables like GROUP_SESSION, INDIVIDUAL_SESSION, PARTICIPATES, and SUPERVISES to represent relationships.

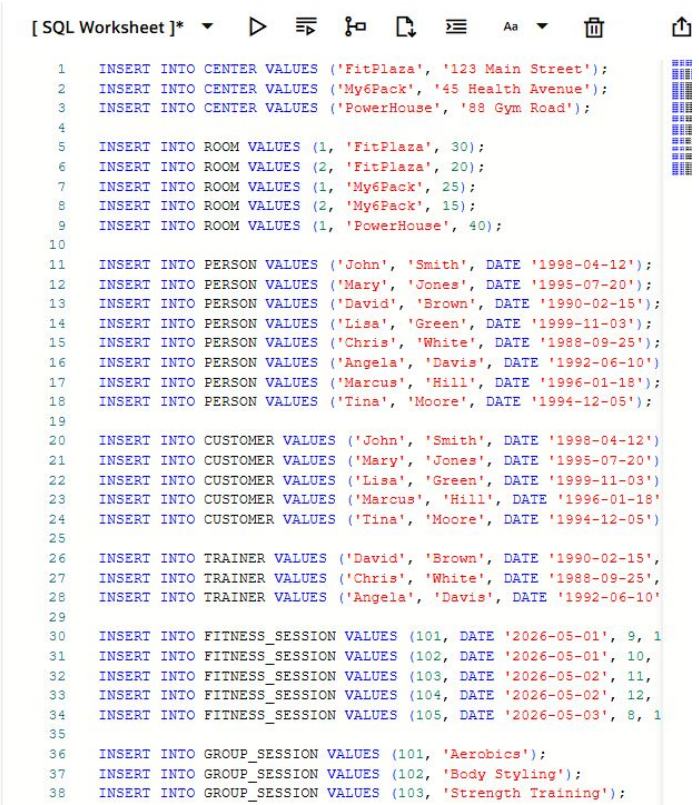
These tables were implemented in Oracle using primary and foreign keys to maintain data integrity.

[SQL Worksheet]*

```sql
CREATE TABLE GROUP_SESSION (
    SessionID NUMBER PRIMARY KEY,
    ClassType VARCHAR2(50),
    FOREIGN KEY (SessionID) REFERENCES SESSION(SessionID)
);

CREATE TABLE INDIVIDUAL_SESSION (
    SessionID NUMBER PRIMARY KEY,
    FOREIGN KEY (SessionID) REFERENCES SESSION(SessionID)
);

CREATE TABLE PARTICIPATES (
    FirstName VARCHAR2(30),
    LastName VARCHAR2(30),
    DateOfBirth DATE,
    SessionID NUMBER,
    PRIMARY KEY (FirstName, LastName, DateOfBirth, SessionID),
    FOREIGN KEY (SessionID) REFERENCES SESSION(SessionID)
);

CREATE TABLE SUPERVISES (
    FirstName VARCHAR2(30),
    LastName VARCHAR2(30),
    DateOfBirth DATE,
    SessionID NUMBER UNIQUE,
    FOREIGN KEY (SessionID) REFERENCES GROUP_SESSION(SessionID)
);
```

```sql
CREATE TABLE CENTER (
    CenterName VARCHAR2(50) PRIMARY KEY,
    Address VARCHAR2(100)
);

CREATE TABLE ROOM (
    RoomNumber NUMBER,
    CenterName VARCHAR2(50),
    Capacity NUMBER,
    PRIMARY KEY (RoomNumber, CenterName),
    FOREIGN KEY (CenterName) REFERENCES CENTER(CenterName)
);

CREATE TABLE PERSON (
    FirstName VARCHAR2(30),
    LastName VARCHAR2(30),
    DateOfBirth DATE,
    PRIMARY KEY (FirstName, LastName, DateOfBirth)
);

CREATE TABLE TRAINER (
    FirstName VARCHAR2(30),
    LastName VARCHAR2(30),
    DateOfBirth DATE,
    Diploma VARCHAR2(50),
    PRIMARY KEY (FirstName, LastName, DateOfBirth),
    FOREIGN KEY (FirstName, LastName, DateOfBirth)
    REFERENCES PERSON(FirstName, LastName, DateOfBirth)
);

CREATE TABLE CUSTOMER (
    FirstName VARCHAR2(30),
    LastName VARCHAR2(30),
    DateOfBirth DATE,
    PRIMARY KEY (FirstName, LastName, DateOfBirth),
    FOREIGN KEY (FirstName, LastName, DateOfBirth)
    REFERENCES PERSON(FirstName, LastName, DateOfBirth)
);
```

Insert Data

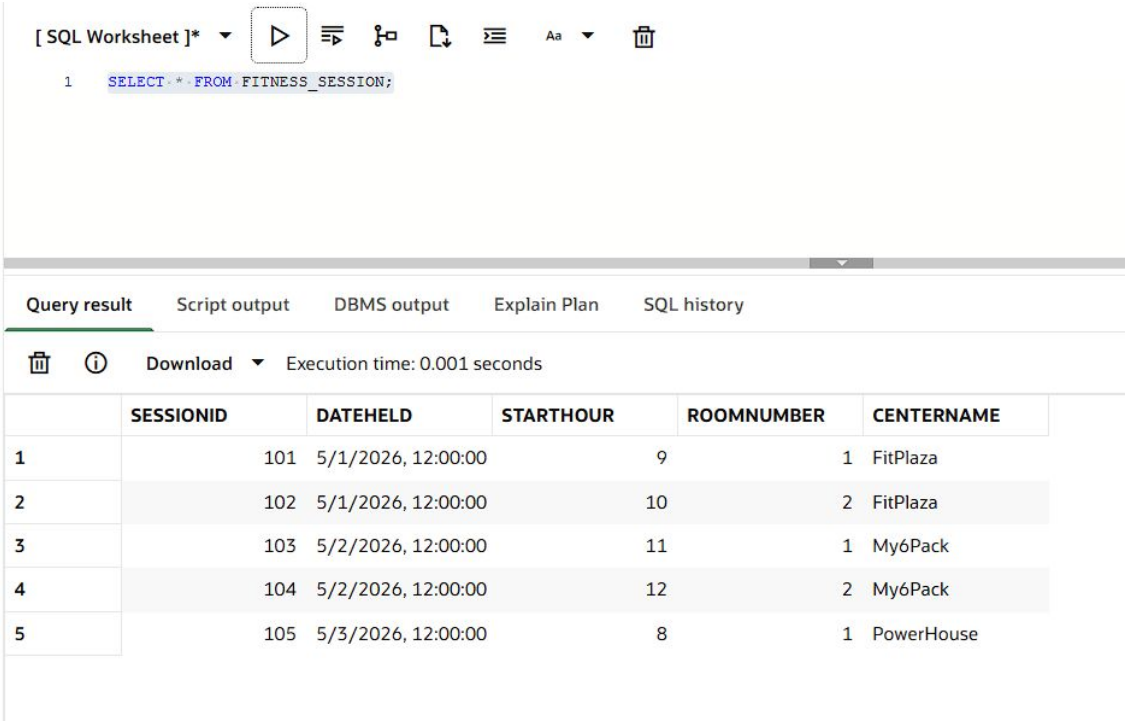


After creating the tables, I inserted data into all tables including CENTER, ROOM, PERSON, CUSTOMER, TRAINER, and FITNESS_SESSION.

I also linked people to sessions using PARTICIPATES and assigned trainers using SUPERVISES.

This step is important because without data, queries would return empty results

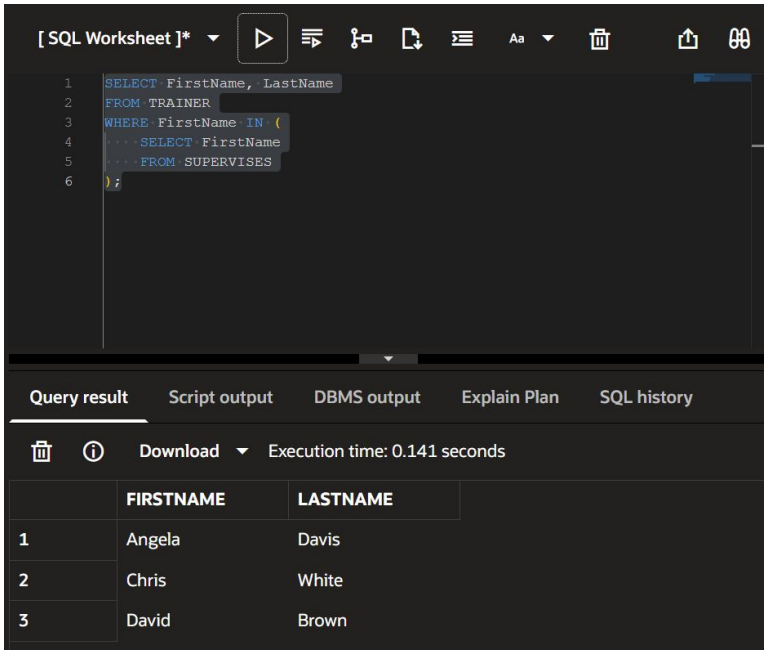
SQL Queries



	SESSIONID	DATEHELD	STARTHOUR	ROOMNUMBER	CENTERNAME
1	101	5/1/2026, 12:00:00	9	1	FitPlaza
2	102	5/1/2026, 12:00:00	10	2	FitPlaza
3	103	5/2/2026, 12:00:00	11	1	My6Pack
4	104	5/2/2026, 12:00:00	12	2	My6Pack
5	105	5/3/2026, 12:00:00	8	1	PowerHouse

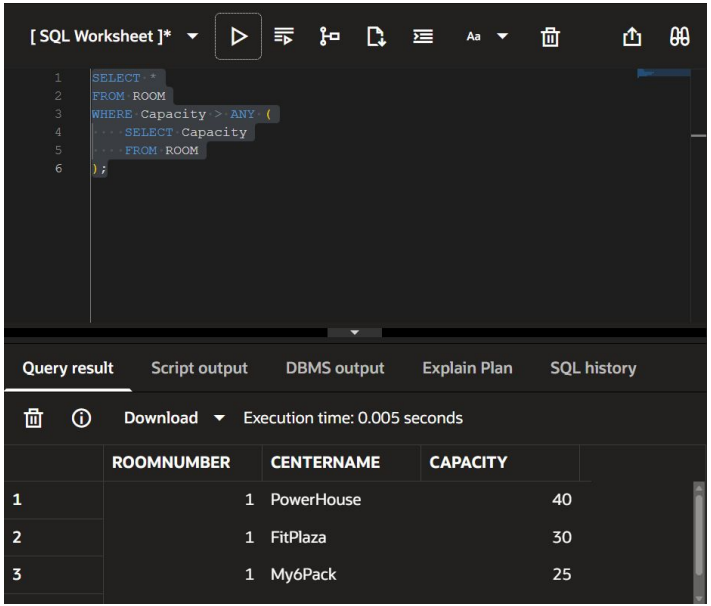
After inserting data, I used SQL queries such as SELECT, JOIN, and aggregation to retrieve and analyze information. I also implemented advanced queries including nested queries, correlated queries, NOT EXISTS, and set operations. For example, a SELECT query retrieves all session data from the database.

Nested query



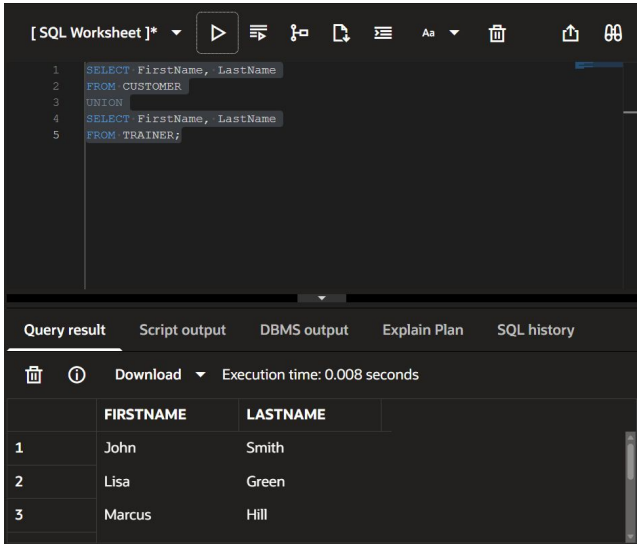
This query uses a nested query to find trainers who supervise sessions. The inner query selects names from the SUPERVISES table, and the outer query returns matching trainers

Any query



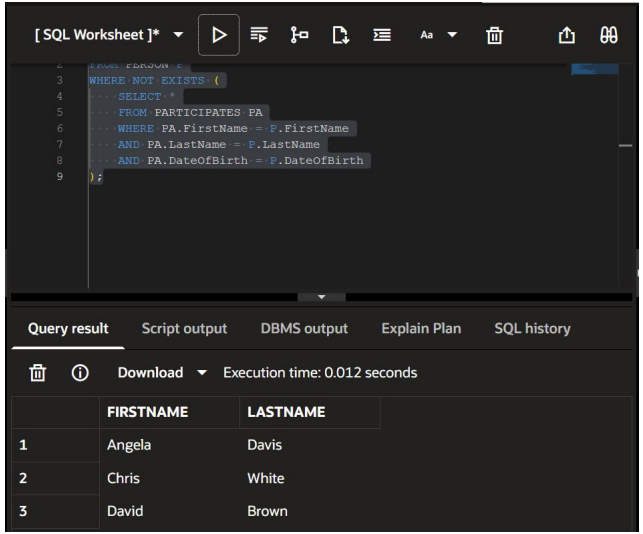
This query returns rooms with a capacity greater than at least one other room. The ANY keyword compares values against a set of results.

UNION (Set Operation)



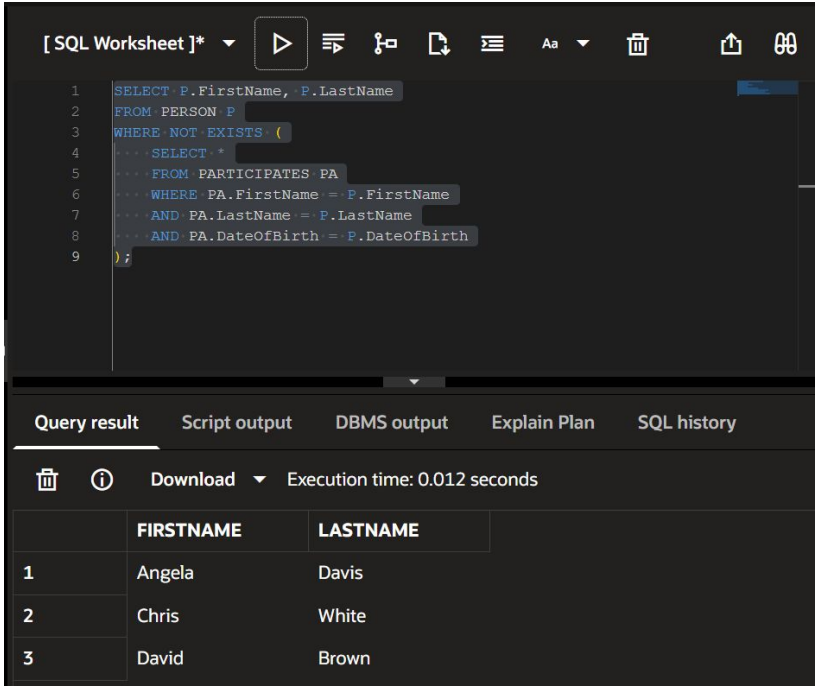
This query combines the results of two queries into one result set. It returns all unique names from both the CUSTOMER and TRAINER tables, removing duplicates.

Correlated Query (EXISTS)



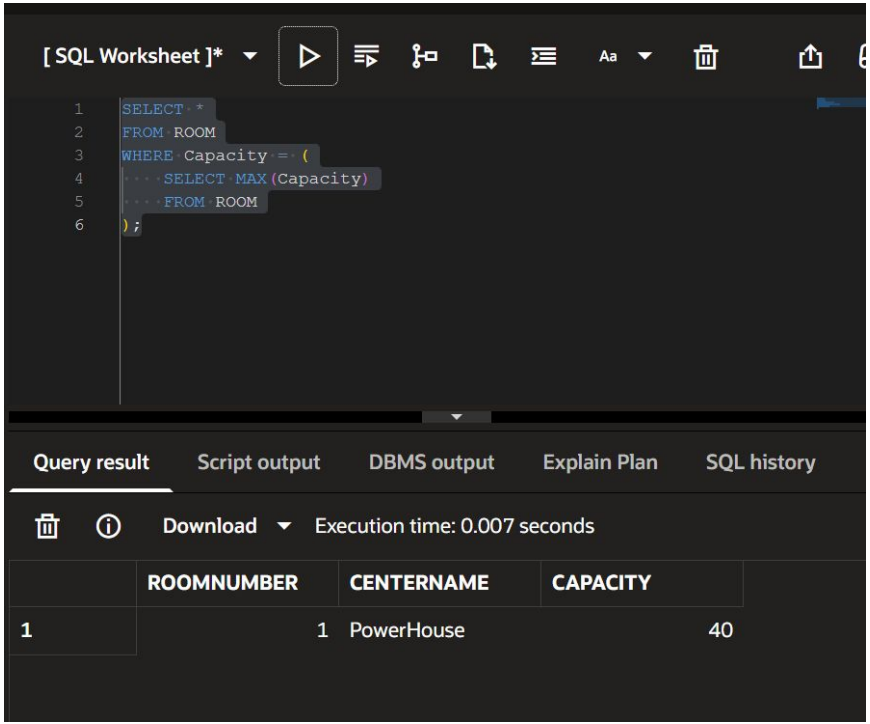
This correlated query checks for each trainer if they exist in the SUPERVISES table. It returns only trainers who are currently supervising sessions.

NOT EXISTS Query



This query finds people who have not participated in any sessions. It uses NOT EXISTS to check for the absence of matching records in the PARTICIPATES table.

Nested Query (MAX)



This query finds the room with the highest capacity by using a nested query with the MAX function.

JOIN Query

[SQL Worksheet]*

```
1  SELECT F.SessionID, F.CenterName, R.Capacity
2  FROM FITNESS_SESSION F
3  JOIN ROOM R
4  ON F.RoomNumber = R.RoomNumber
5  AND F.CenterName = R.CenterName;
```

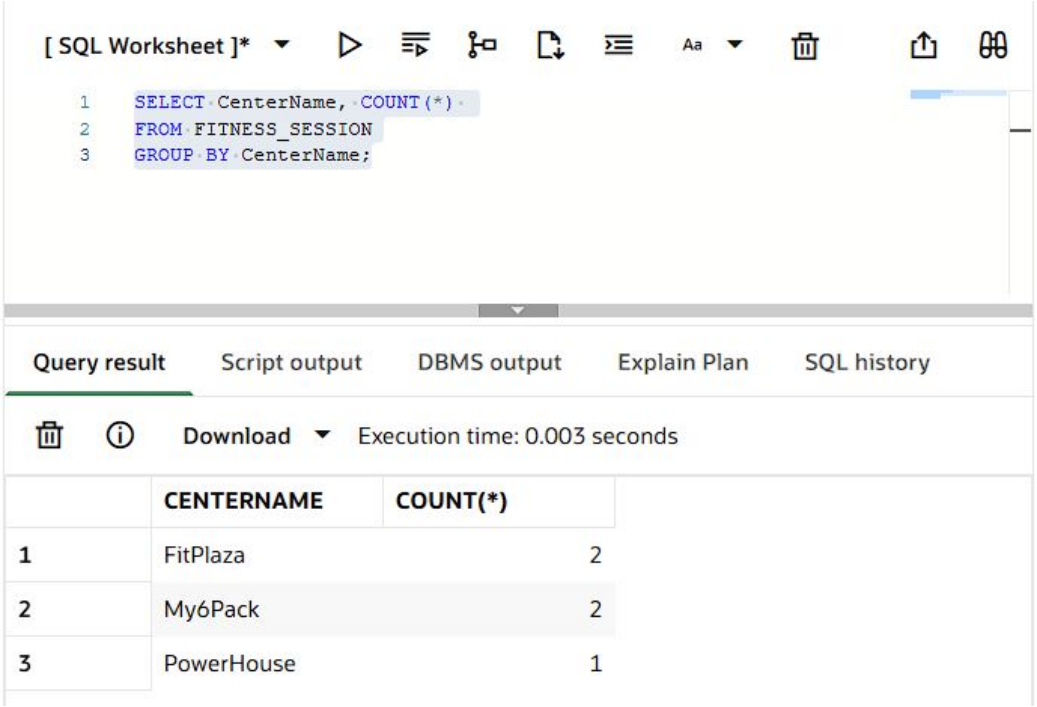
Query result | Script output | DBMS output | Explain Plan | SQL hi

Download | Execution time: 0.011 seconds

	SESSIONID	CENTERNAME	CAPACITY
1	101	FitPlaza	30
2	102	FitPlaza	20
3	103	My6Pack	25
4	104	My6Pack	15
5	105	PowerHouse	40

This JOIN query combines data from the FITNESS_SESSION and ROOM tables. It uses matching room number and center name to connect the tables. The result shows each session along with the room capacity.

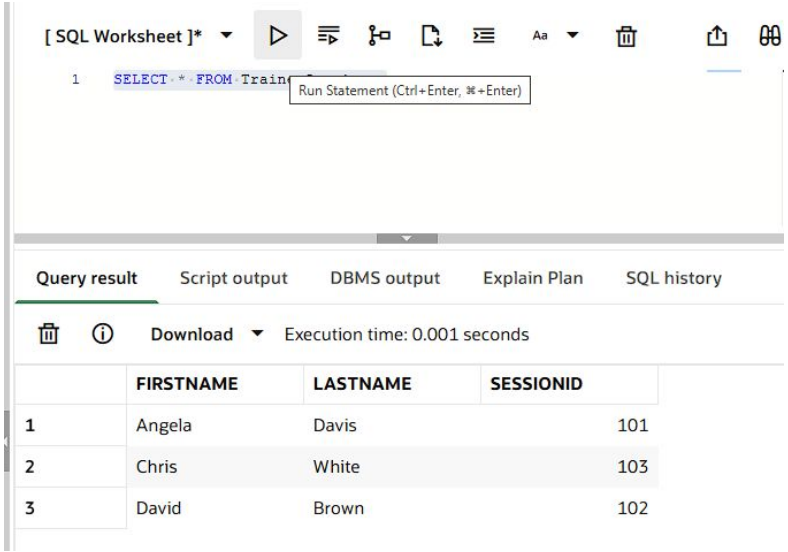
Aggregation Query



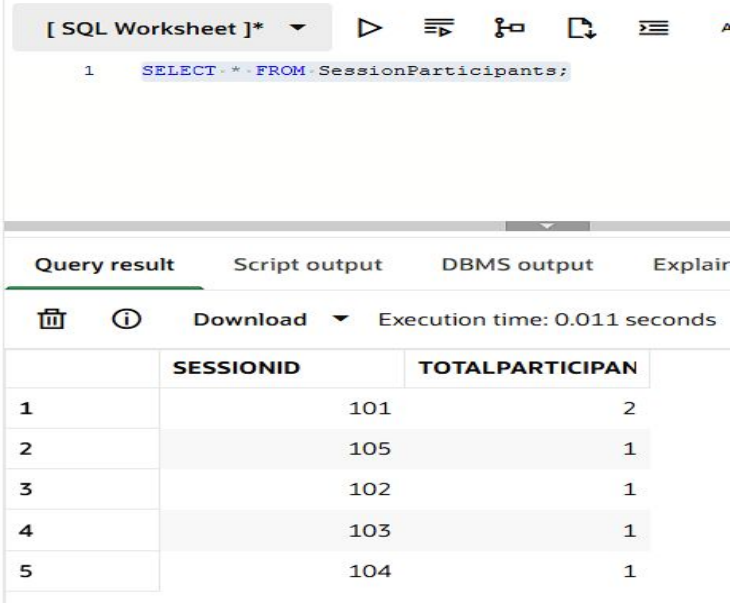
This aggregation query uses COUNT and GROUP BY to summarize the data. It shows how many sessions are held at each center. For example, FitPlaza and My6Pack each have two sessions, while PowerHouse has one.

Views (Virtual Tables)

Views simplify complex queries by storing them as reusable virtual tables. I created views such as TrainerSessions and SessionParticipants to make it easier to access information about trainers, sessions, and participants. I also created a CenterSessions view to summarize the total number of sessions at each center.

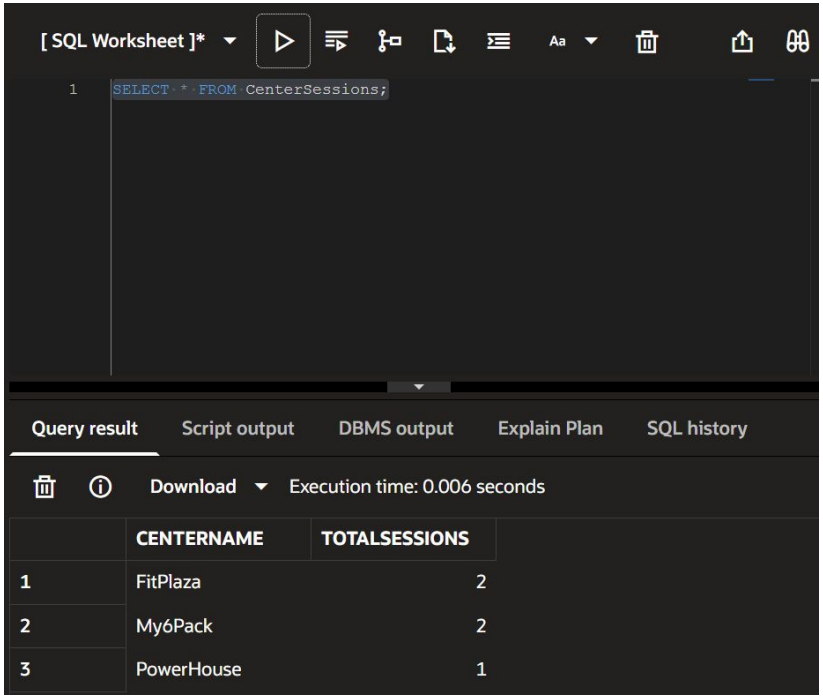


	FIRSTNAME	LASTNAME	SESSIONID
1	Angela	Davis	101
2	Chris	White	103
3	David	Brown	102

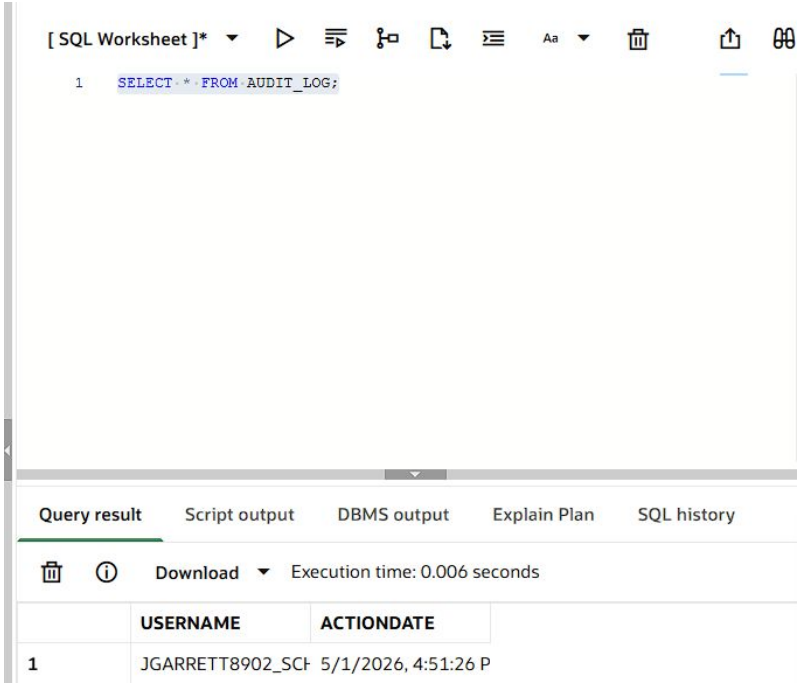


	SESSIONID	TOTALPARTICIPAN
1	101	2
2	105	1
3	102	1
4	103	1
5	104	1

Views


[SQL Worksheet]*
1
SELECT * FROM CenterSessions;
Query result
Script output
DBMS output
Explain Plan
SQL history
Download
Execution time: 0.006 seconds
CENTERNAME
TOTALSESSIONS
1
FitPlaza
2
2
My6Pack
2
3
PowerHouse
1

Trigger (Audit Logging) challenge



This screenshot shows the result of my trigger. After updating the customer table, the trigger automatically inserted a record into the audit log table. It stores the username and the date and time of the update.

Conclusion

In conclusion, this project demonstrates how databases can be used to efficiently manage real-world systems. I designed and implemented a complete database, applied ER modeling and relational schema, and used SQL to store, retrieve, and analyze data.

Overall, this project strengthened my understanding of database design and its real-world applications.